

Documento Técnico — Sistema de Evaluación de Infraestructura Física

Universidad del Norte

Versión: 2.0

Fecha: 23 de julio de 2026

Elaborado por: Daniel Gutiérrez - Jefe de Estadística y Analítica Institucional

Destinatario: Dirección de Tecnología Informática

1. Resumen Ejecutivo

El **Sistema de Evaluación de Infraestructura Física** es una aplicación web diseñada para que 39 evaluadores realicen inspecciones presenciales de 495 espacios físicos del campus, calificando cada espacio mediante una rúbrica estandarizada de 5 dimensiones (escala 1-5) con soporte para evidencia fotográfica.

● IMPORTANTE

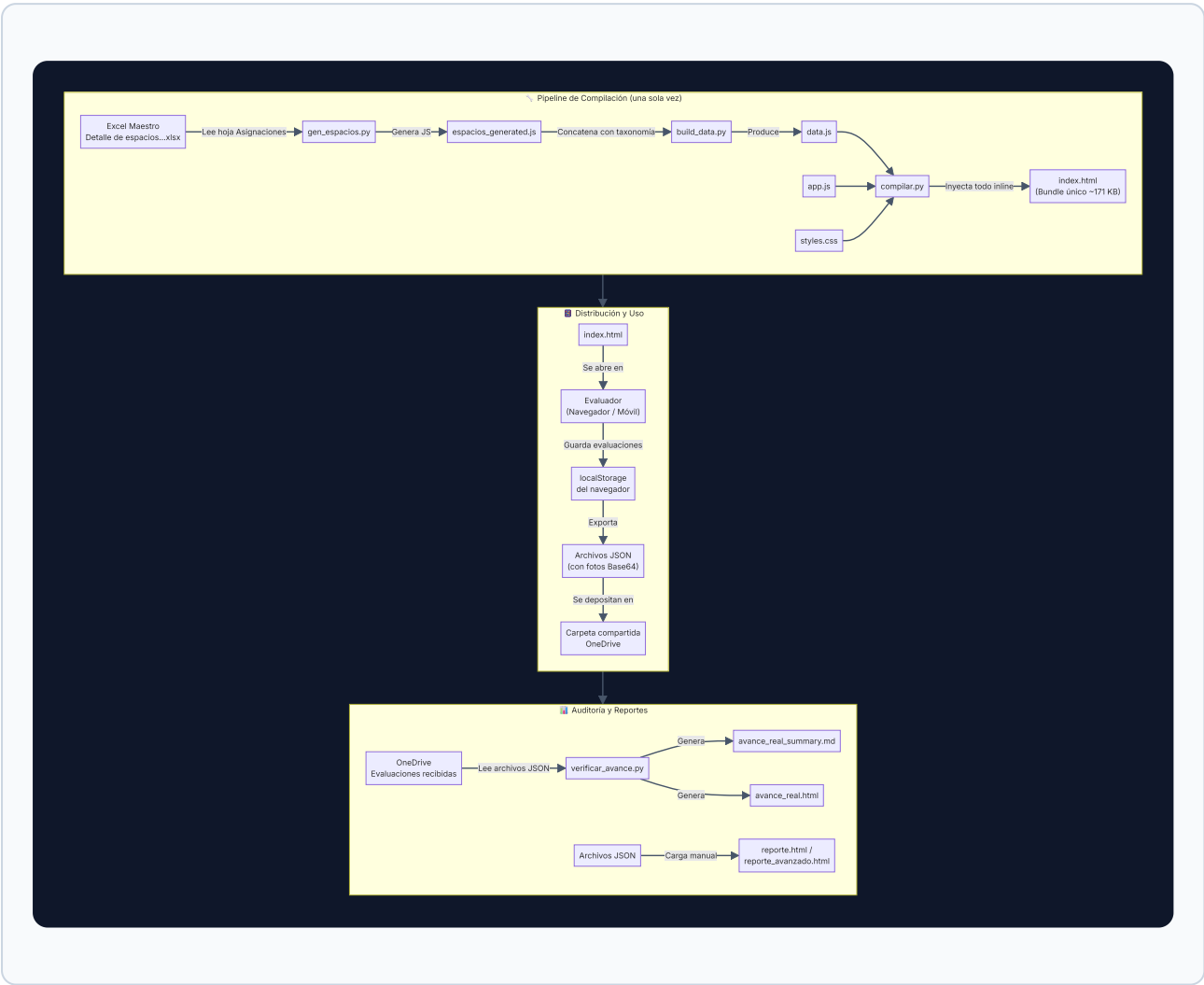
La aplicación **no requiere servidor backend, base de datos ni conexión a internet** para funcionar. Es una aplicación 100% del lado del cliente (client-side) que opera de manera offline en cualquier navegador moderno.

Componentes del sistema

Componente	Descripción	Tecnología
App de Evaluación	Formulario de captura de evaluaciones con fotos	HTML5 + JavaScript + CSS (archivo único)
Reportes Analíticos	Dashboards para visualizar resultados	HTML5 + Chart.js
Pipeline de Compilación	Scripts que ensamblan la app a partir de sus fuentes	Python 3.x
Auditoría de Avance	Script que cruza evaluaciones recibidas vs. asignadas	Python 3.x + openpyxl

2. Arquitectura del Sistema

2.1 Diagrama General de Arquitectura



2.2 Tipo de Arquitectura

Aspecto	Detalle
Paradigma	Single Page Application (SPA) client-side, zero-server
Backend requerido	✗ Ninguno
Base de datos	✗ Ninguna — usa <code>localStorage</code> del navegador
API REST	✗ No existe
Servidor web	Solo para servir archivos estáticos (IIS, Apache, Nginx, o incluso <code>file://</code>)
Autenticación actual	✗ No implementada
Conexión a internet	Solo para cargar la fuente tipográfica (Google Fonts) y Chart.js (CDN). Luego funciona offline

3. Stack Tecnológico

3.1 Frontend (Aplicación de Evaluación)

Tecnología	Versión	Uso
HTML5	—	Estructura de la aplicación
JavaScript ES6+	Vanilla	Lógica de formularios, cascadas de selección, compresión de fotos, almacenamiento local, exportación
CSS3	Custom Properties, Flexbox, Grid	Diseño responsivo con tema oscuro/claro y efectos glassmórficos
Google Fonts	Inter	Tipografía (cargada desde CDN <code>fonts.googleapis.com</code>)
Canvas API	HTML5	Compresión de imágenes a 800px max y 50% calidad JPEG

3.2 Reportes y Dashboards

Tecnología	Versión	Uso
Chart.js	4.x	Gráficas de dona y barras (cargado desde <code>cdn.jsdelivr.net</code>)
HTML5 Drag & Drop API	—	Carga de archivos JSON por arrastrar y soltar

3.3 Pipeline de Compilación y Auditoría (Python)

Tecnología	Versión	Uso
Python	3.10+	Scripts de generación, compilación y auditoría
openpyxl	3.x	Lectura del archivo Excel maestro de asignaciones
Módulos estándar	os , re , json , csv , unicodedata , datetime	Procesamiento de datos

4. Inventario de Archivos

4.1 Archivos de Producción (distribución a usuarios)

Archivo	Tamaño	Descripción
index.html	171 KB	App principal. Bundle único autocontenido con CSS, JS y datos incrustados. Es el único archivo que necesitan los evaluadores
icon.png	479 KB	Ícono de la aplicación (iOS Add to Home Screen)
reporte.html	38 KB	Dashboard analítico estándar para visualizar evaluaciones
reporte_avanzado.html	45 KB	Dashboard analítico avanzado con controles de privacidad y exclusión de espacios

4.2 Archivos Fuente (desarrollo y compilación)

Archivo	Tamaño	Descripción
app.js	44 KB	Lógica principal de la aplicación (1,303 líneas)
data.js	91 KB	Diccionario maestro: 495 espacios, 39 evaluadores, 12 categorías con rúbricas
styles.css	22 KB	Estilos CSS con temas oscuro/claro
compilar.py	3 KB	Script que inyecta CSS, data.js y app.js dentro de index.html
gen_espacios.py	3 KB	Generador de datos desde Excel
build_data.py	1 KB	Ensamblador de data.js

4.3 Archivos de Datos y Auditoría

Archivo	Tamaño	Descripción
Detalle de espacios físicos para rúbrica v3.0.xlsx	75 KB	Archivo maestro de asignaciones (fuente de verdad)
espacios_generated.js	59 KB	Salida intermedia del generador
espacios.json	68 KB	Representación JSON de los espacios
verificar_avance.py	36 KB	Script de auditoría que cruza evaluaciones vs. asignaciones
avance_real.html	135 KB	Dashboard interactivo de avance (generado por verificar_avance.py)

4.4 Documentación

Archivo	Descripción
Guía rápida del Evaluador.pdf	Manual de usuario para evaluadores
Rúbrica de evaluación de infraestructura física.pdf	Documento de referencia de la rúbrica
guia_rapida_evaluador.md	Guía rápida en formato Markdown
rubrica_infraestructura_uninorte.md	Especificación técnica de la rúbrica

5. Flujo de Datos Detallado

5.1 Fase 1 — Preparación de Datos (una sola vez)

Excel Maestro (.xlsx)

- [gen_espacios.py] Lee la hoja "Asignaciones"
 - └─ Extrae: Categoría, Edificio, Código, Piso, Nombre, Evaluador
 - └─ Mapea categorías Excel → IDs internos (cat_aulas, cat_labs, etc.)
 - └─ Genera: espacios_generated.js (array ESPACIOS + lista EVALUADORES)
- [build_data.py] Concatena espacios_generated.js + taxonomía de rúbricas
 - └─ Genera: data.js (diccionario maestro completo)
- [compilar.py] Inyecta styles.css + data.js + app.js dentro de index.html
 - └─ Genera: index.html (bundle autocontenido listo para distribución)

5.2 Fase 2 — Evaluación en Campo (repetitiva)

Evaluador abre index.html en su dispositivo

- Selecciona su nombre → Se filtran los espacios asignados
- Selecciona Categoría → Edificio → Código (cascada automática)
- El sistema auto-rellena Nombre y Piso del espacio
- Califica 5 dimensiones (escala 1-5 con descriptores)
- Captura hasta 2 fotos por dimensión
 - └ Las fotos se comprimen via Canvas API (800px max, 50% JPEG)
 - └ Se almacenan como cadenas Base64 dentro del objeto de evaluación
- Agrega notas/observaciones opcionales
- La evaluación se guarda en localStorage del navegador
 - └ Clave: "uninorte_evaluaciones_v2"
 - └ Límite estimado: ~5 MB (con indicador visual de capacidad)

5.3 Fase 3 — Exportación y Recolección

Evaluador exporta sus evaluaciones

- Formato JSON: Incluye metadatos + calificaciones + fotos Base64
- Formato CSV: Solo calificaciones numéricas (sin fotos)
- El archivo JSON se deposita en la carpeta compartida de OneDrive:
 - └ "Evaluaciones recibidas/"

5.4 Fase 4 — Auditoría y Reportes

Administrador ejecuta: `python verificar_avance.py`

- Lee el Excel maestro → Construye mapa de asignaciones por evaluador
- Escanea carpeta OneDrive → Lee todos los archivos JSON
- Cruza evaluaciones vs. asignaciones:
 - └ Evaluaciones completadas (match por código de espacio)
 - └ Evaluaciones pendientes (asignadas pero no encontradas)
 - └ Evaluaciones extras (realizadas pero no estaban asignadas)
- Genera: `avance_real_summary.md` (reporte Markdown)
- Genera: `avance_real.html` (dashboard interactivo con filtros y ordenamiento)

6. Almacenamiento y Gestión de Datos

6.1 Almacenamiento en el Cliente

Aspecto	Detalle
Mecanismo	<code>window.localStorage</code>
Clave	<code>uninorte_evaluaciones_v2</code>
Formato	JSON serializado (array de objetos de evaluación)
Capacidad típica	~5 MB por origen (límite del navegador)
Persistencia	Persiste entre sesiones, se pierde si el usuario limpia datos del navegador
Fallback	Si <code>localStorage</code> no está disponible (ej. iOS WebKit en modo privado), usa un almacén en memoria volátil
Indicador visual	Barra de capacidad con alertas al 80% y 90%

6.2 Estructura de un Registro de Evaluación (JSON)

```
{
  "evaluador": "NOMBRE DEL EVALUADOR",
  "fecha": "2026-07-15T10:30:00",
  "categoria": "cat_aulas",
  "categoriaName": "Salón de clases",
  "edificio": "Bloque A",
  "codigo": "SAL-101",
  "piso": "1",
  "nombre": "Salón 101",
  "dimensiones": {
    "dim1": { "score": 4, "fotos": ["data:image/jpeg;base64, ..."], "notas": "" },
    "dim2": { "score": 3, "fotos": [], "notas": "Observación ..." },
    "dim3": { "score": 5, "fotos": [], "notas": "" },
    "dim4": { "score": 4, "fotos": ["data:image/jpeg;base64, ..."], "notas": "" },
    "dim5": { "score": 3, "fotos": [], "notas": "" }
  },
  "promedio": 3.8,
  "clasificacion": "Bueno"
}
```

6.3 Fotos Embebidas

Aspecto	Detalle
Formato	JPEG comprimido via Canvas API
Resolución máxima	800 × 800 píxeles
Calidad	50%
Almacenamiento	Cadenas Base64 incrustadas dentro del JSON
Capacidad	Hasta 2 fotos por dimensión, 5 dimensiones = máx. 10 fotos por espacio
Tamaño estimado por foto	~30-80 KB en Base64

7. Dependencias Externas (CDN)

⚠ ADVERTENCIA

La aplicación actualmente depende de dos recursos externos cargados desde CDNs públicos. Para un despliegue en infraestructura universitaria controlada, se recomienda internalizar estos recursos.

Recurso	URL	Uso	Criticidad
Google Fonts (Inter)	<code>https://fonts.googleapis.com/css2?family=Inter</code>	Tipografía de la interfaz	Baja — el navegador usa fuente del sistema como fallback
Chart.js 4.x	<code>https://cdn.jsdelivr.net/npm/chart.js</code>	Gráficas en dashboards de reportes	Media — solo afecta <code>reporte.html</code> y <code>reporte_avanzado.html</code>

Cómo Internalizar

1. Descargar el archivo CSS de Google Fonts y las fuentes `.woff2` asociadas.
2. Descargar `chart.js` desde [jsdelivr](https://www.jsdelivr.com/) o [npmjs](https://www.npmjs.com/).
3. Alojar ambos en el servidor estático de la universidad.
4. Actualizar las referencias `<link>` y `<script src>` en los archivos HTML.

8. Opciones de Despliegue

8.1 Opción A — Hosting de Archivos Estáticos (Recomendado)

CONSEJO

Esta es la opción más simple y adecuada dado que la aplicación es 100% client-side.

Requisitos:

- Servidor web que sirva archivos estáticos (IIS, Apache, Nginx)
- Sin procesamiento server-side requerido
- Sin base de datos

Estructura sugerida en el servidor:

```
/rubrica-evaluacion/
├── index.html           ← App principal (171 KB)
├── icon.png             ← Ícono (479 KB)
├── reporte.html         ← Dashboard estándar
├── reporte_avanzado.html ← Dashboard avanzado
├── fonts/               ← Google Fonts internalizadas
│   └── inter.woff2
├── libs/                ← Librerías internalizadas
│   └── chart.js
├── docs/                ← Documentación
│   ├── guia_evaluador.pdf
│   └── rubrica.pdf
```

Configuración IIS (Windows Server):

```
<!-- web.config -->
<configuration>
  <system.webServer>
    <staticContent>
      <mimeTypeMap fileExtension=".json" mimeType="application/json" />
      <mimeTypeMap fileExtension=".woff2" mimeType="font/woff2" />
    </staticContent>
    <defaultDocument>
      <files>
        <add value="index.html" />
      </files>
    </defaultDocument>
  </system.webServer>
</configuration>
```

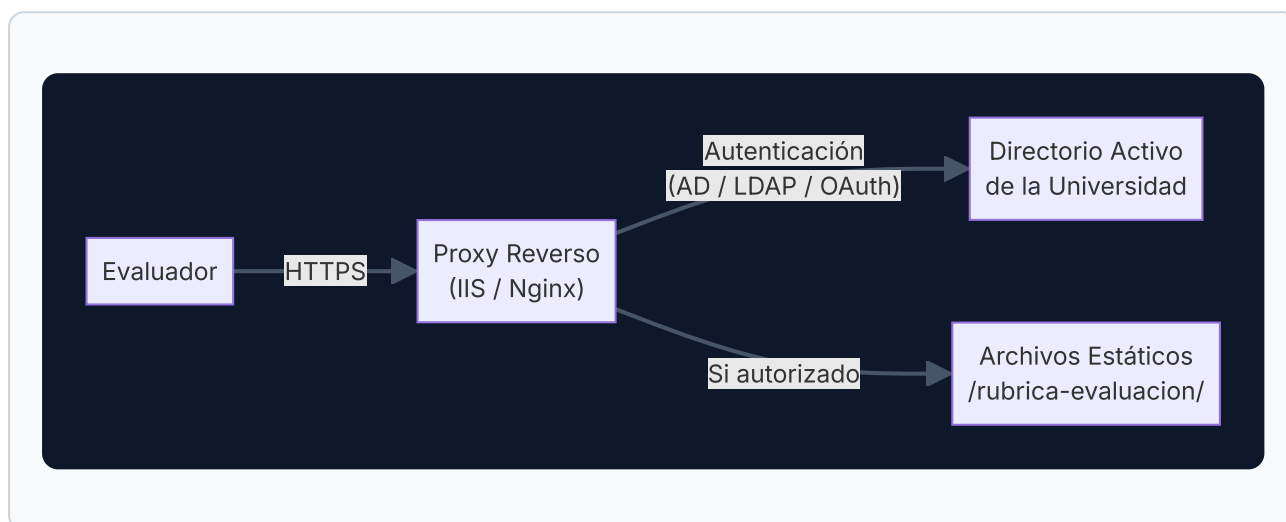
Recursos estimados:

Recurso	Mínimo
:---	:---
CPU	Negligible (solo sirve archivos estáticos)
RAM	< 50 MB

| Disco | < 5 MB (archivos de la app) |
| Ancho de banda | ~200 KB por carga inicial por usuario |

8.2 Opción B — Hosting con Proxy Reverso y Autenticación

Si se requiere control de acceso, se puede colocar un proxy reverso (como IIS con Windows Authentication o un reverse proxy con Nginx + LDAP/OAuth) delante del directorio estático.



Opciones de autenticación:

Método	Complejidad	Descripción
Windows Authentication (IIS)	Baja	Usa las credenciales de dominio del usuario. Transparente para usuarios en la red universitaria
Basic Auth (Nginx/Apache)	Baja	Lista de usuarios y contraseñas en archivo <code>.htpasswd</code>
OAuth2 / OpenID Connect	Media	Integración con proveedor de identidad institucional (Azure AD, Google Workspace, etc.)
LDAP Bind	Media	Autenticación directa contra el directorio LDAP/AD de la universidad

NOTA

Dado que la aplicación es 100% client-side, la autenticación solo protege el **acceso al archivo `index.html`**. Una vez descargado, el HTML opera de manera autónoma en el navegador del usuario. Esto es una característica de diseño, no una vulnerabilidad, ya que permite el uso offline.

8.3 Opción C — Despliegue como Progressive Web App (PWA)

La aplicación ya incluye metadatos para "Add to Home Screen" en iOS. Para habilitarla como PWA completa se necesitaría:

1. Crear `manifest.json` (actualmente no existe):

```
{
  "name": "Evaluación de Infraestructura - UniNorte",
  "short_name": "Rúbrica UniNorte",
  "start_url": "/rubrica-evaluacion/index.html",
  "display": "standalone",
  "background_color": "#0a0e1a",
  "theme_color": "#6366f1",
  "icons": [
    { "src": "icon.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

1. Crear **Service Worker** para cacheo offline (actualmente no existe).

2. **HTTPS obligatorio** — los Service Workers solo funcionan sobre HTTPS.

9. Consideraciones de Seguridad y Gobierno de Datos

9.1 Estado Actual de Seguridad

Aspecto	Estado Actual	Riesgo
Autenticación de usuarios	✗ No implementada	Medio — cualquiera con el archivo puede usarlo
Cifrado de datos en reposo	✗ No (localStorage en texto plano)	Bajo — datos no sensibles (infraestructura, no personas)
Cifrado en tránsito	Depende del hosting (HTTPS)	Medio — recomendable HTTPS
Validación de integridad de datos	✗ No hay checksums ni firmas digitales	Bajo — los JSON pueden ser editados manualmente
Control de acceso a reportes	✗ No implementado	Medio — cualquiera puede abrir los dashboards
Auditoría de accesos	✗ No existe	Bajo — se puede lograr con logs del servidor web
Respaldo de datos	Depende de OneDrive	Bajo — OneDrive tiene versionado

9.2 Recomendaciones de Seguridad

● IMPORTANTE

Las siguientes recomendaciones están ordenadas por prioridad e impacto.

Prioridad Alta

1. **Servir sobre HTTPS** — Configurar certificado SSL/TLS en el servidor web para cifrar la transmisión del HTML y evitar interceptación.
2. **Autenticación a nivel de servidor web** — Implementar Windows Authentication en IIS (si los evaluadores están en dominio) o Basic Auth con lista controlada de usuarios.
3. **Internalizar CDNs** — Descargar Google Fonts y Chart.js para alojarlos localmente, eliminando dependencia de servicios externos y riesgos de inyección por CDN comprometido.

Prioridad Media

1. **Crear `manifest.json` y Service Worker** — Habilitar la experiencia PWA completa con cacheo offline controlado.
2. **Agregar cabeceras de seguridad HTTP:**

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline';
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

1. **Logs de acceso** — Habilitar logging en el servidor web para auditar quién accede a la aplicación y cuándo.

Prioridad Baja

1. **Checksums de archivos JSON exportados** — Agregar un hash SHA-256 al JSON exportado para detectar manipulación posterior.
2. **Versiónado de la app** — Agregar un número de versión visible en el frontend para controlar qué versión usan los evaluadores.

9.3 Datos Personales y Privacidad

Tipo de Dato	Presente	Clasificación
Nombres de evaluadores	✓ Sí	Dato personal básico
Fotos de espacios físicos	✓ Sí	No personal (infraestructura)
Calificaciones numéricas	✓ Sí	Dato institucional
Credenciales de acceso	✗ No existen	N/A
Datos de estudiantes/pacientes	✗ No	N/A

NOTA

El sistema **no procesa datos sensibles** (salud, financieros, académicos). Los únicos datos personales son los nombres completos de los evaluadores, que son empleados de la universidad actuando en capacidad institucional. El `reporte_avanzado.html` ya incluye un control de privacidad que permite ocultar nombres de evaluadores en los reportes.

10. Requisitos de Infraestructura

10.1 Para el Servidor de Hosting

Requisito	Especificación
Sistema Operativo	Windows Server (IIS) o Linux (Nginx/Apache)
Servidor Web	IIS 10+, Nginx 1.18+, o Apache 2.4+
Espacio en Disco	< 5 MB para los archivos de la aplicación
RAM	Mínimo asignado al servidor web (< 100 MB)
CPU	Negligible — solo sirve archivos estáticos
Red	Acceso desde la red universitaria (intranet o internet según política)
HTTPS	Certificado SSL/TLS (requerido para PWA, recomendado en general)
Base de datos	✗ No requiere
Runtime (Node, PHP, .NET)	✗ No requiere ninguno

10.2 Para la Estación de Administración (Scripts Python)

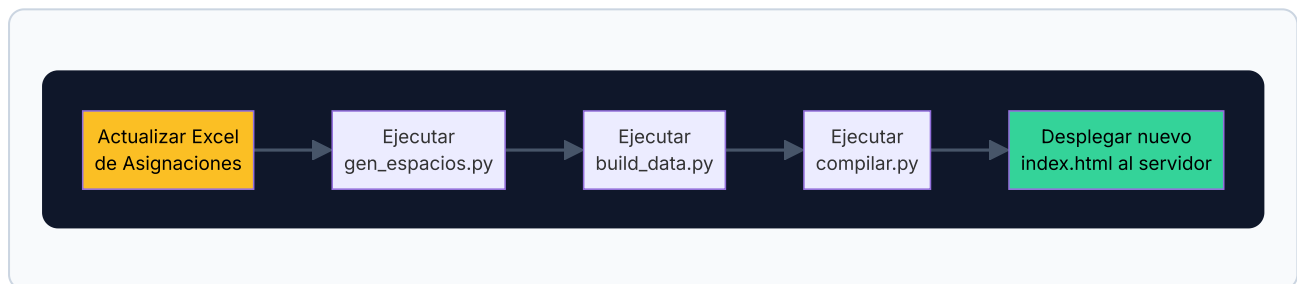
Requisito	Especificación
Python	3.10 o superior
Librería	<code>openpyxl</code> (<code>pip install openpyxl</code>)
Acceso a OneDrive	Lectura de la carpeta <code>Evaluaciones recibidas</code> sincronizada localmente
Acceso al Excel Maestro	Detalle de espacios físicos para rúbrica <code>v3.0.xlsx</code>

10.3 Para los Dispositivos de los Evaluadores

Requisito	Especificación
Navegador	Chrome 90+, Safari 14+, Firefox 90+, Edge 90+
Cámara	Para captura de evidencia fotográfica
Conectividad	Solo necesaria para la descarga inicial de <code>index.html</code> . Después funciona 100% offline
Almacenamiento	~5 MB de <code>localStorage</code> disponible en el navegador

11. Procedimiento de Actualización

Cuando se modifica el Excel maestro de asignaciones o la rúbrica, el proceso de actualización es:



Comandos:

```
cd C:\ruta\al\proyecto\rubrica-uninorte-v2

# 1. Regenerar datos desde Excel
python gen_espacios.py

# 2. Ensamblar data.js
python build_data.py

# 3. Compilar bundle final
python compilar.py

# 4. Copiar index.html al servidor web
copy index.html \\servidor\wwwroot\rubrica-evaluacion\
```

12. Preguntas Frecuentes Técnicas

¿Se necesita instalar algo en los dispositivos de los evaluadores?

No. Solo necesitan abrir `index.html` en un navegador moderno. No hay instalación de apps nativas.

¿Qué pasa si un evaluador borra el caché del navegador?

Pierde las evaluaciones almacenadas localmente. Por eso es fundamental que exporten sus evaluaciones a JSON regularmente.

¿Se pueden recuperar datos si el evaluador pierde su teléfono?

Solo si previamente exportó sus evaluaciones a un archivo JSON. No hay respaldo centralizado automático.

¿Es posible centralizar el almacenamiento de evaluaciones en un servidor?

Sí, pero requeriría desarrollar un backend (API REST) para recibir y almacenar evaluaciones. Esto cambiaría significativamente la arquitectura actual. Algunas opciones:

- API REST con Node.js/Express + base de datos PostgreSQL o MongoDB
- Azure Functions + Cosmos DB (serverless)
- SharePoint Lists API (aprovechando la infraestructura Microsoft existente)

¿Se puede integrar con el sistema de identidad de la universidad (SSO)?

Sí, a nivel de servidor web. La aplicación en sí no necesita modificaciones — el control de acceso se implementa en la capa del servidor.

¿Cuál es el riesgo de que alguien modifique un archivo JSON de evaluación?

Actualmente no hay validación de integridad. Un usuario técnico podría editar el JSON. Para mitigar esto, se podría implementar un hash de verificación en futuras versiones.

13. Contacto y Soporte

Rol	Responsable
Administrador funcional	Dirección de Planeación y Estudios Institucionales
Soporte técnico de despliegue	Dirección de Tecnología Informática
Repositorio de código	[GitHub — repositorio del proyecto]

CONSEJO

Para una demostración en vivo del sistema, se puede abrir `index.html` directamente en un navegador sin ninguna instalación previa. Los dashboards de reporte (`reporte.html` , `reporte_avanzado.html`) pueden cargarse con el archivo de prueba `test_datos_ejemplo.json` incluido en el repositorio.